V4 Feature Engineering Process

V4 เป็น generated/synthetic end-to-end pipeline ใช้ XGBoost + SMOTE + Optuna

Input / Output

Item	Path / Value
Input clean data	data\processed\clean_dataset.csv
Engineered dataset	docs\version 4\data\features\df_engineered_v4_generated.csv
Used feature count	180
Code files	docs\version 4\scripts\clean_dataset_v4.py docs\version 4\scripts\run_v4_generated_end_to_end_pipeline.py

Feature Engineering Steps

1. Generate synthetic order/return data เพื่อจำลอง imbalance data
2. Clean data และสร้าง clean_dataset_v4_generated.csv
3. ทำ EDA: target distribution, category, channel, payment และ correlation heatmap
4. สร้าง point-in-time aggregate features เช่น customer/category/brand/province/payment/channel/courier return rates
5. สร้าง one-hot encoded features และ feature interactions
6. ใช้ SMOTE สำหรับ imbalanced data
7. Train LogisticRegression, RandomForest, XGBoost, LightGBM และ tune ด้วย Optuna
8. Evaluate ด้วย Accuracy, Recall, F1, AUC, Cost Matrix และ SHAP

Feature Groups Created / Used

Feature group	Count
product_category	64
payment_channel	38
customer_profile	26
customer_history	18
promotion_discount	13
logistics	9
price_amount	5
time	4
other	3

Reasoning

V4 เหมาะเป็น showcase end-to-end pipeline เพราะ Accuracy สูง แต่เป็น generated data และ cost สูงกว่า V2 จึงไม่ใช่ production winner บนข้อมูลจริง

Model Result Snapshot

Metric	Value
Model	XGBoost_SMOTE_Optuna
Accuracy	83.45%
Recall	46.39%
Precision	45.00%

Metric	Value
F1	45.69%
AUC	85.38%
Cost	31,650
Rating	C

Pseudo-code / Code Logic

```
raw = generate_synthetic_orders_returns()
clean = clean_dataset_v4(raw)
features = add_point_in_time_aggregates(clean)
features = add_one_hot_interactions(features)
X_train, X_test, y_train, y_test = train_test_split(features, target, stratify=target)
X_train_smote, y_train_smote = SMOTE().fit_resample(X_train, y_train)
model = tune_xgboost_with_optuna(X_train_smote, y_train_smote)
evaluate_cost_auc_shap(model, X_test, y_test)
```